

QUESTION BANK(HPC)

MODULE 1

MAY 2024

PART A

1. Explain the principle of locality.

The principle of locality in high-performance computing states that programs perform best when they repeatedly access the same data (temporal locality) and access data stored close together in memory (spatial locality). By maximizing locality, HPC applications reduce memory access latency, improve cache efficiency, and minimize costly data movement, leading to higher performance and scalability.

2. State Amdahl's law.

Amdahl's Law states that the maximum speedup of a parallel program is limited by the fraction of the program that must be executed serially. Even with an infinite number of processors, the serial portion restricts the overall performance improvement.

PART B

3. Discuss in detail the importance of considering processor performance for the design of an efficient computer system.

- Processor performance is a key factor in the design of an efficient computer system because it directly determines how fast instructions are executed and how well the system can meet application requirements. A well-designed processor ensures optimal use of hardware resources and overall system efficiency.
- Firstly, **instruction execution speed** is crucial. Factors such as clock speed, instruction-level parallelism, pipelining, and multicore architectures influence how many instructions can be completed per second. High processor performance reduces execution time and improves system responsiveness.
- Secondly, **efficient utilization of memory hierarchy** depends on processor design. Modern processors rely on cache memory (L1, L2, L3) to reduce memory access latency. A processor optimized for cache usage minimizes delays caused by slow main memory, improving throughput.

- Thirdly, **parallel processing capability** is essential. Multicore processors and support for multithreading allow multiple tasks to execute simultaneously. This improves performance for parallel applications and enhances system scalability, especially in high-performance and server environments.
 - Fourthly, **power efficiency and thermal management** must be considered. High-performance processors consume more power and generate heat. Efficient processor design balances performance with power consumption to ensure reliability, lower operating costs, and longer hardware lifespan.
 - Fifthly, **system balance** is important. Processor performance must be matched with memory bandwidth, I/O subsystems, and interconnects. A fast processor with slow memory or I/O leads to bottlenecks and underutilized resources.
 - Finally, considering processor performance enables **cost-effective system design**. Selecting the appropriate processor architecture avoids overdesign or underperformance, ensuring the system meets performance goals within budget constraints.
 - Considering processor performance is essential for achieving high speed, efficiency, scalability, power optimization, and cost-effectiveness in computer system design.
4. **Discuss the role played by virtual machine in providing protection for process.**

Role of Virtual Machines in Providing Process Protection

- **Isolation of Processes**
Virtual machines (VMs) provide strong isolation by running each process or group of processes in a separate virtual environment, preventing one process from directly accessing another's memory or resources.

- **Memory Protection**

VMs use virtual memory and address translation to ensure that a process can access only its allocated memory space, protecting the system from accidental or malicious memory access.

- **Controlled Resource Access**

Hardware resources such as CPU, storage, and I/O devices are accessed through the virtual machine monitor (hypervisor), which enforces access permissions and prevents unauthorized use.

- **Fault Containment**

Errors or crashes within a process or VM are contained within that virtual environment and do not affect other processes or the host system.

- **Security Enforcement**

Virtual machines support security policies such as access control, sandboxing, and privilege separation, reducing the risk of system compromise.

- **Safe Execution of Untrusted Code**

VMs allow untrusted or experimental applications to run safely without endangering the host operating system or other processes.

- **Simplified Recovery and Management**

Features like snapshots and rollback help restore processes to a safe state after failure, enhancing reliability and protection.

Virtual machines play a crucial role in protecting processes by providing isolation, controlled access, fault tolerance, and enhanced security.

5. Explain in detail the instruction set architecture.

The **Instruction Set Architecture (ISA)** is the programmer-visible interface between **hardware and software**. It defines how software controls the processor and how instructions are executed, independent of the internal hardware implementation.

- **Instruction Set**

ISA specifies the set of machine instructions supported by the processor, such as arithmetic, logical, data transfer, control flow, and system instructions. These determine what operations the processor can perform.

- **Instruction Formats**

It defines the layout of instructions, including opcode fields, operand fields, addressing mode bits, and instruction length. This ensures correct decoding and execution of instructions.

- **Addressing Modes**

ISA specifies how operands are accessed (e.g., immediate, register, direct, indirect, indexed). Multiple addressing modes increase flexibility and programming efficiency.

- **Registers**

The ISA defines the number, size, and purpose of registers (general-purpose registers, special registers like program counter, stack pointer, and status flags).

- **Data Types and Sizes**

It specifies supported data types (integers, floating-point, characters) and word sizes, which affect precision, memory usage, and performance.

- **Memory Model**

ISA defines how memory is organized and accessed, including byte ordering (endianness), alignment requirements, and virtual memory support.

- **Exception and Interrupt Handling**

It specifies how the processor responds to interrupts, traps, and exceptions, enabling safe execution and interaction with the operating system.

The instruction set architecture provides a stable and well-defined interface that allows software portability while enabling different hardware implementations to achieve performance, power, and cost goals.

6)What is Pipelining? Explain major hurdles of pipelining with the help of an example

Pipelining

- **Pipelining** is a technique in which the execution of an instruction is divided into several stages, and multiple instructions are executed simultaneously, each in a different stage of the pipeline.
- It improves **instruction throughput** without reducing the execution time of a single instruction.

Major Hurdles of Pipelining (with Examples)

1. Structural Hazards

- Occur when two or more pipeline stages need the same hardware resource at the same time.
- Caused by insufficient hardware resources.
- **Example:**

Using a single memory for both instruction fetch and data access leads to conflicts.

2. Data Hazards

- Occur when an instruction depends on the result of a previous instruction that has not yet completed.
- The most common type is **Read After Write (RAW)**.
- **Example:**
- ADD R1, R2, R3
- SUB R4, R1, R5

The SUB instruction needs the result of ADD, causing a pipeline stall.

3. Control Hazards

- Occur due to branch instructions that change the flow of program execution.
- The pipeline may fetch incorrect instructions before the branch outcome is known.

- **Example:**

BEQ R1, R2, LABEL

- If the branch is taken, already fetched instructions must be discarded.
- Pipelining increases processor performance but is limited by structural, data, and control hazards.
- These hurdles are handled using techniques like **resource duplication, forwarding, stalling, and branch prediction.**

OCT 2023

PART A

1) Define the concept of principle of locality. (Repeated)

- **Definition:** The principle of locality states that programs tend to reuse data and instructions that were accessed recently.
- **Types:**
 - **Temporal locality** – Recently accessed items are likely to be accessed again soon.
 - **Spatial locality** – Items stored near each other in memory are likely to be accessed close together in time.
- **Importance:** This property is exploited in computer design through **caches and memory hierarchies** to improve performance.
- **Rule of thumb:** A program spends about **90% of its execution time in only 10% of the code.**

2) What is the equation of CPI for pipelined processor? How can we set the ideal pipelined CPI?

CPI Equation for Pipelined Processor

- **CPI (Cycles Per Instruction)** is defined as:

$$\left[CPI = \frac{\text{CPU clock cycles for a program}}{\text{Instruction count}} \right]$$

- For a **pipelined processor**, the CPU time is:

$$[CPU\ Time = Instruction\ Count \times CPI \times Clock\ Cycle\ Time]$$

Ideal Pipelined CPI

- In an **ideal pipeline** (no stalls, hazards, or delays):
 - **One instruction completes every clock cycle.**
 - Hence, **CPI = 1.**
- Due to hazards (structural, data, control) and stalls, the actual CPI is **greater than 1.**

3) Explain in detail the instruction set architecture.(repeated)

Instruction Set Architecture (ISA)

- **Definition:**
- The ISA is the **programmer-visible interface** between hardware and software. It specifies the instructions a processor can execute, how they are encoded, and how data is accessed. It acts as the **boundary between compilers/programs and hardware.**

Dimensions of ISA

1. Class of ISA

- Most modern ISAs are **general-purpose register architectures.**
- Two types:
 - Register-Memory ISAs** (e.g., x86) → can access memory directly in instructions.
 - Load-Store ISAs** (e.g., ARM, RISC-V) → memory accessed only via load/store instructions.

2. Memory Addressing

- Most ISAs use **byte addressing.**
- Alignment rules: ARM requires aligned access, while x86 allows unaligned but slower.

3. Addressing Modes

- Ways to specify operand locations.

- b. Examples: Register, Immediate, Displacement, PC-relative, Autoincrement/Autodecrement.
- c. x86 supports complex modes like scaled index + displacement.

4. Operand Types and Sizes

- a. Common sizes: 8-bit, 16-bit, 32-bit, 64-bit integers.
- b. Floating-point: IEEE 754 single (32-bit) and double (64-bit).
- c. x86 also supports 80-bit extended precision.

5. Operations

- a. Categories: Data transfer, Arithmetic, Logical, Control flow, Floating-point.
- b. Example: RISC-V has simple arithmetic and load/store operations for easy pipelining.

6. Control Flow Instructions

- a. Conditional branches, jumps, procedure calls, returns.
- b. Usually use **PC-relative addressing**.
- c. Example: ARM and RISC-V store return address in a register; x86 uses stack.

7. Instruction Encoding

- a. **Fixed length** (ARM, RISC-V → 32-bit instructions, easier decoding).
- b. **Variable length** (x86 → 1–18 bytes, compact but harder to decode).

Importance of ISA

- Defines **performance, energy efficiency, and programmability**.
- Influences **compiler design, hardware complexity, and instruction decoding**.
- Acts as a **contract between software developers and hardware designers**.

ISA specifies the **instructions, addressing modes, operand types, operations, control flow, and encoding** of a processor. It is the **bridge between hardware and software**, shaping performance, efficiency, and ease of programming.

4) What is Pipelining? Explain major hurdles of pipelining with the help of an example.(repeated)

Pipelining

- **Definition:**
- Pipelining is a technique used in processors to **overlap the execution of instructions** by dividing the instruction cycle into stages (Fetch, Decode, Execute, Memory, Write-back).

- Each stage works in parallel on different instructions, so ideally **one instruction completes every clock cycle**.
- **Goal:** Improve throughput (instructions per unit time) without reducing the execution time of a single instruction.

Major Hurdles of Pipelining

1. Structural Hazards

- Occur when hardware resources are insufficient to support all pipeline stages simultaneously.
- Example: If both instruction fetch and data memory access need the same memory unit, a conflict arises.

2. Data Hazards

- Occur when an instruction depends on the result of a previous instruction that has not yet completed.
- Types:
 - RAW (Read After Write)** – e.g., ADD R1, R2, R3 followed immediately by SUB R4, R1, R5.
 - WAR (Write After Read)**
 - WAW (Write After Write)**
- Example: If instruction 2 needs the result of instruction 1 before it is written back, the pipeline must stall.

3. Control Hazards (Branch Hazards)

- Occur due to branch or jump instructions that change the flow of execution.
- Example: If a branch is taken, instructions already fetched may be invalid, causing wasted cycles.
- Solution: Branch prediction and speculative execution.

Example

Consider two instructions:

- LOAD R1, 100(R2)
- ADD R3, R1, R4

- In a pipeline, the **ADD** instruction tries to use R1 before the **LOAD** has completed.
- This is a **data hazard** → the pipeline must stall or use forwarding to resolve it.

Pipelining improves throughput by overlapping instruction execution. The major hurdles are **structural hazards (resource conflicts)**, **data hazards (dependency issues)**, and **control hazards (branching problems)**. Example: LOAD followed by ADD causes a data hazard.

5) Discuss in detail the importance of considering processor performance for the design of an efficient computer system. *(repeated)*

- Processor performance refers to how quickly and efficiently a CPU executes instructions. It is measured using metrics like **CPU time**, **CPI (Cycles per Instruction)**, **clock cycle time**, and **instruction count**.
- **Formula:**

$$CPU\ Time = Instruction\ Count \times CPI \times Clock\ Cycle\ Time$$

where:

- **Instruction Count (IC):** Number of instructions executed.
- **CPI (Cycles Per Instruction):** Average number of cycles per instruction.
- **Clock Cycle Time:** Duration of one clock cycle (inverse of clock rate).

Why Processor Performance Matters

1. Overall System Efficiency

- a. The processor is the **core component** of a computer system.
- b. Faster processors reduce execution time, improving responsiveness for both users and applications.

2. Throughput and Scalability

- a. In servers and high-performance systems, processor performance determines how many tasks or transactions can be handled per second.
- b. Efficient processors allow scaling to larger workloads without excessive cost.

3. Energy Efficiency

- a. Performance must be balanced with **power consumption**.
- b. Example: Mobile devices require processors that deliver high performance while conserving battery life.

4. Cost-Effectiveness

- a. A processor with better performance per cost unit ensures **price-performance optimization**.
- b. Important in desktops, laptops, and warehouse-scale computers.

5. Support for Parallelism

- a. Modern workloads demand **parallel execution** (data-level, task-level, thread-level).
- b. Processor performance determines how well pipelining, superscalar execution, and multicore architectures can be exploited.

6. Dependability and Availability

- a. High performance processors reduce delays in critical systems (e.g., banking servers, airline reservations).
- b. Ensures **availability and reliability** under heavy loads.

7. Impact on Memory Hierarchy

- a. Processor speed must be matched with efficient **cache and memory design**.
- b. If processor performance is ignored, memory bottlenecks will waste CPU cycles.

8. Quantitative Design Principles

- a. Metrics like **Amdahl's Law** and the **Processor Performance Equation** guide designers in balancing instruction count, CPI, and clock cycle time.
- b. Example: Even a faster clock won't help if CPI is high due to pipeline stalls.

Example

- In a **web server**, processor performance directly affects how many requests can be served per second.
- If the CPU is slow, even with large memory and storage, the system will fail to meet throughput requirements.

Considering processor performance is **essential** for designing efficient computer systems. It impacts **speed, scalability, energy efficiency, cost, dependability, and memory interaction**. Ignoring it leads to bottlenecks and poor system design, while optimizing it ensures balanced, high-performance computing across desktops, servers, mobile devices, and supercomputers.

6) Suppose that we want to enhance the processor used for Web server. The new processor is 10 times faster on computation in the Web serving application than the original processor. Assuming that the original processor is busy with computation 40% of the time and is waiting for I/O 60% of the time, what is the overall speedup gained by incorporating the enhancement?

- Computation time fraction = **40% = 0.40**
- I/O wait time fraction = **60% = 0.60**
- Speedup of computation = **10×**

We need to calculate the **overall speedup** using **Amdahl's Law**.

Formula (Amdahl's Law)

$$\text{Overall Speedup} = \frac{1}{(1-f) + \frac{f}{S}}$$

Where:

- f = fraction of time enhanced (here, computation = 0.40)
- S = speedup of enhanced part (here, 10)

$$\begin{aligned}\text{Overall Speedup} &= \frac{1}{(1 - 0.40) + \frac{0.40}{10}} \\ &= \frac{1}{0.60 + 0.04} \\ &= \frac{1}{0.64} \\ &= 1.56\end{aligned}$$

The **overall speedup = 1.56 times**.

Even though computation is made **10× faster**, the processor spends **60% of its time waiting for I/O**, which cannot be improved. Hence, the maximum overall speedup is limited to **1.56×**, showing the practical limitation of Amdahl's Law.

MAY 2024

PART A

1. Differentiate between data level parallelism and task level parallelism

- **Data-Level Parallelism (DLP):**
 - Same operation performed on **multiple data items simultaneously**.
 - Example: Vector processors, SIMD instructions, GPUs.
 - Focus: Exploiting parallelism in **large datasets**.
- **Task-Level Parallelism (TLP):**
 - Different **independent tasks** executed in parallel.
 - Example: Multithreading, MIMD architectures.
 - Focus: Exploiting parallelism in **separate tasks or processes**.

2. Describe the encoding scheme specified in MIPS ISA

- **MIPS uses fixed-length instruction encoding.**

- All instructions are **32 bits long**, which simplifies decoding and pipelining.

- **Instruction Formats:**

- **R-Format (Register type):** Used for register-to-register operations.
 - Fields: opcode | rs | rt | rd | shamt | funct
- **I-Format (Immediate type):** Used for data transfer, branch, and immediate instructions.
 - Fields: opcode | rs | rt | immediate
- **J-Format (Jump type):** Used for jump instructions.
 - Fields: opcode | address

- **Floating-point instructions** also have their own formats but follow the same fixed-length principle.

3. Describe the quantitative principle of computer design with Amdahl's law.

Computer architects use quantitative principles to design efficient systems. The key principles are:

1. Take Advantage of Parallelism

- a. Performance improves when tasks are executed in parallel.
- b. Examples: pipelining, multicore processors, vector units, caches.

2. Principle of Locality

- a. Programs reuse data and instructions recently accessed.
- b. Two types:
 - i. **Temporal locality** → recently used data is likely to be used again.
 - ii. **Spatial locality** → nearby memory locations are likely to be accessed together.
- c. This principle justifies the use of **caches**.

3. Focus on the Common Case

- a. Optimize the most frequent operations rather than rare ones.
- b. Example: arithmetic operations are optimized more than overflow handling.

Amdahl's Law

- **Definition:**

Amdahl's Law states that the performance improvement from an enhancement is limited by the fraction of time the enhancement can be used.

- **Formula:**

$$Speedup = \frac{1}{(1 - f) + \frac{f}{S}}$$

Where:

- f = fraction of execution time affected by the enhancement
- S = speedup of the enhanced portion
- **Explanation:**
 - If only a small fraction of the program benefits from the enhancement, the overall speedup will be small, even if the enhancement is very fast.
 - Example: If 40% of execution time is improved by 10×, overall speedup is only 1.56×.

Importance in Computer Design

- Guides architects to **quantify trade-offs** in performance, cost, and energy.
- Shows that **parallelism and locality** must be exploited broadly, not just in small parts of the system.
- Helps avoid over-investing in improvements that have little impact on overall performance.

4. We are considering an enhancement to the processor, What will be the overall speed, if you make 10% of a program 90 times faster?

- Fraction of program enhanced = **10% = 0.10**
- Speedup of enhanced portion = **90×**
- Remaining fraction (not enhanced) = **90% = 0.90**

Formula (Amdahl's Law)

$$\text{Overall Speedup} = \frac{1}{(1 - f) + \frac{f}{S}}$$

Where:

- f = fraction of execution time enhanced

- S = speedup of enhanced part

$$\begin{aligned}\text{Overall Speedup} &= \frac{1}{(1 - 0.10) + \frac{0.10}{90}} \\ &= \frac{1}{0.90 + 0.0011} \\ &= \frac{1}{0.9011} \\ &\approx 1.11\end{aligned}$$

The **overall speedup = 1.11 times**.

5. Illustrate how processes are protected with the help of virtual memory

Protection of Processes Using Virtual Memory

Virtual memory is a powerful mechanism that provides process protection by isolating address spaces and managing access to memory. Virtual memory ensures system stability, prevents unauthorized memory access, and isolates processes to achieve fault tolerance and security.

Illustration of Virtual Memory in Process Protection

Example: Consider two processes, Process A and Process B, running simultaneously on a system with virtual memory:

1. Memory Isolation:

- Process A has virtual address space 0x0000 to 0xFFFF mapped to physical memory pages 0x1000 to 0x1FFF.
- Process B has virtual address space 0x0000 to 0xFFFF mapped to physical memory pages 0x2000 to 0x2FFF.
- If Process A tries to access address 0x2000 (belonging to Process B), the MMU will block the access and raise an exception.

2. Protection Bits Example:

- Process A's page table marks page 0x1000 as read-only.
- If Process A attempts to write to page 0x1000, the MMU raises a fault, and the operating system prevents unauthorized modification.

3. Page Fault Handling:

- Process B tries to access a page not currently in memory.

- The MMU raises a page fault, and the operating system retrieves the required page from disk, ensuring that only Process B's memory is affected.

6. Explain different classes of computer

CLASSES OF COMPUTERS

The changes in computer use have led to five diverse computing markets, each characterized by different applications, requirements, and computing technologies. The mainstream classes of computing environments and their important characteristics are:

Internet of Things/Embedded Computers

- Embedded computers are found in everyday machines: microwaves, washing machines, most printers, networking switches, and all automobiles.
- Internet of Things (IoT) refers to embedded computers that are connected to the Internet, typically wirelessly.
- Embedded computers have the widest spread of processing power and cost.
- They include 8-bit to 32-bit processors that may cost one penny, and high-end 64-bit processors for cars and network switches that cost \$100.
- Although the range of computing power in the embedded computing market is very large, price is a key factor in the design of computers for this space.

Personal Mobile Device

- Personal mobile device (PMD) is the term we apply to a collection of wireless devices with multimedia user interfaces such as cell phones, tablet computers, and so on.
- Cost is a prime concern given the consumer price for the whole product is a few hundred dollars.
- Although the emphasis on energy efficiency is frequently driven by the use of batteries, the need to use less expensive packaging plastic versus ceramic—and the absence of a fan for cooling also limit total power consumption.
- The processors in a PMD are often considered embedded computers, but we are keeping them as a separate category because PMDs are platforms that can run externally developed software, and they share many of the characteristics of Desktop computers
- Responsiveness and predictability are key characteristics for media applications. ➤ A real-time performance requirement means a segment of the application has an absolute maximum execution time

- Other key characteristics in many PMD applications are the need to minimize memory and the need to use energy efficiently.
- Energy efficiency is driven by both battery power and heat dissipation

Desktop Computing

- Desktop computing spans from low-end netbooks that sell for under \$300 to high end, heavily configured workstations that may sell for \$2500.
- Since 2008, more than half of the desktop computers made each year have been battery operated laptop computers. Desktop computing sales are declining
- Throughout this range in price and capability, the desktop market tends to be driven to optimize price-performance.
- Desktop computing also tends to be reasonably well characterized in terms of applications and benchmarking, though the increasing use of web-centric, interactive applications pose new challenges in performance evaluation.

Servers

- For servers, different characteristics are important. First, availability is critical
- A second key feature of server systems is scalability.
- Server systems often grow in response to an increasing demand for the services they support or an expansion in functional requirements.
- Thus the ability to scale up the computing capacity, the memory, the storage, and the I/O bandwidth of a server is crucial.
- Finally, servers are designed for efficient throughput. That is, the overall performance of the server—in terms of transactions per minute or web pages served per second—is what is crucial.
- Responsiveness to an individual request remains important, but overall efficiency and cost-effectiveness, as determined by how many requests can be handled in a unit time, are the key metrics for most servers.

Clusters/Warehouse-Scale Computers

- Clusters are collections of desktop computers or servers connected by local area networks to act as a single larger computer.
- Each node runs its own operating system, and nodes communicate using a networking protocol.
- WSCs are the largest of the clusters, in that they are designed so that tens of thousands of servers can act as one.
- Price-performance and power are critical to WSCs since they are so large.

- WSCs are related to servers in that availability is critical
- Supercomputers are related to WSCs in that they are equally expensive, costing hundreds of millions of dollars, but supercomputers differ by emphasizing floating-point performance and by running large, communication-intensive batch programs that can run for weeks at a time